Filtered Approximate Nearest Neighbor Search on Vectors with Diverse Labels: The Supplement

1 PSEUDO CODE OF ApproximateELS

The approximated ELS retrieval method (ApproximateELS, Algorithm 1) initially checks whether $L_q \in F$ (Lines 2-6). If $L_q \in F$, it returns $\hat{F}_q = \{L_q\}$ (Line 8). Otherwise, the algorithm enumerates the nodes in $\mathbb{I}(l_{\max})$, where $l_{\max}$ is the queried label with the largest ID (Lines 9-10). Each such node is treated as a possible anchor on a trie path that may contain all labels in L_q. The function BottomUp-Traversal then walks from the anchor toward the root and matches query labels in reverse ID order (Lines 12-21). During this traversal, ancestor labels with IDs larger than the current target label are skipped, a matched label advances the query pointer, and the traversal is pruned once a smaller label or the root is reached before all query labels are matched (Lines 16–21). If all query labels are matched, the anchor is regarded as a valid starting point for relaxed ELSs (Lines 22-23).

Finally, CollectCandidateELSs performs a breadth-first traversal from the validated anchor node and inserts every encountered terminal label set into $\hat{F}_q^{(a)}$ (Lines 24-32). Note that, since each label set in F corresponds to a unique terminal node in the trie and $\hat{F}_q^{(a)}$ is maintained as a set, repeated collections of the same label set, if any, are automatically removed by the insert operation. Unlike exact ELS retrieval, ApproximateELS does not further filter these candidates to retain only minimal supersets of L_q. Therefore, the returned set may contain non-minimal supersets, but each collected terminal label set remains compatible with the query-label containment condition.

Time complexity of ApproximateELS**.** Lines 1–8 check whether L_q itself is a terminal label set in the trie tree, which takes $O(|L_q|)$ time. Then, Lines 9–10 enumerate all nodes in $\mathbb{I}(l_{\max})$, and Lines 13–21 perform a bottom-up traversal from each enumerated node. Since each trie path has length at most $|\mathcal{L}|$, this part costs

$$O\big(|\mathcal{L}| \cdot |\mathbb{I}(l_{\max})|\big). \tag{1}$$

For every validated anchor node, Lines 24–32 collect candidate ELSs by downward traversals. Let Υ denote the number of such downward traversal paths. Since each path costs at most $O(|\mathcal{L}|)$, this part costs

$$O\big(|\mathcal{L}| \cdot \Upsilon\big). \tag{2}$$

Therefore, the total time complexity of ApproximateELS is

$$O\Big(|\mathcal{L}| \cdot \big(|\mathbb{I}(l_{\max})| + \Upsilon\big)\Big), \tag{3}$$

where the $O(|L_q|)$ checking cost is dominated by the trie traversal cost. Compared with the exact ELS retrieval method, ApproximateELS skips the minimal-superset filtering step, and therefore has a smaller time complexity.

Algorithm 1 ApproximateELS

Input: trie tree T with root r, inverted list $\mathbb{I}$, and the queried label set L_q
Output: the set $\hat{F}_q^{(a)}$ of approximated ELSs

```
1:  F̂_q^(a) ← ∅, current node c ← root r of T
2:  for each queried label l ∈ L_q in the increasing order of IDs do
3:      if no child of c corresponds to l then
4:          break
5:      else
6:          c ← child of c that corresponds to l
7:  if c is a terminal node that corresponds to L_q then
8:      return F̂_q^(a) = {L_q}
9:  for each node u ∈ I(l_max) do
10:     BottomUp-Traversal(L_q, u)
11: return F̂_q^(a)
12: function BottomUp-Traversal(L_q, u)
13:     u_initial ← u, i ← |L_q| − 1
14:     while i ≥ 0 do
15:         u_p ← parent of u on the trie tree
16:         if u_p matches a label with a larger ID than L_q[i] then
17:             u ← u_p
18:         else if u_p matches L_q[i] then
19:             u ← u_p, i ← i − 1
20:         else if u_p matches a label with a smaller ID than L_q[i] or u_p is r then
21:             break
22:     if i < 0 then
23:         CollectCandidateELSs(u_initial)
24: function CollectCandidateELSs(u_i)
25:     Q ← {u_i}
26:     while Q ≠ ∅ do
27:         v ← Q.dequeue()
28:         if v is a terminal node corresponding to L ∈ F then
29:             F̂_q^(a).insert(L)
30:         else
31:             for each child x of v on T do
32:                 Q.enqueue(x)
```

2 THEOREM AND PROOF

Theorem 1. *For every label set $L \in \hat{F}_q^{(a)}$ such that $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}$, by employing at least one entry vector whose associated label set is L, the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones.*

Proof. First, for any $S \subseteq F$, we use $Desc^+(S)$ to represent the group of label sets that can be reached by a traversal of LNG starting from nodes that correspond to label sets inside S. Since $\hat{F}_q$ is the set of minimum supersets of L_q on LNG, we have

$$Desc^+(\hat{F}_q) = F_{pass}. \tag{4}$$

Moreover, since $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}$, we have

$$Desc^+(\hat{F}_q) \subseteq Desc^+(\hat{F}_q^{(a)}) \subseteq Desc^+(F_{pass}). \tag{5}$$

Furthermore, given that edges on LNG indicate label set containment relations, we have

$$Desc^+(F_{pass}) = F_{pass}. \tag{6}$$

The above equations indicate that

$$Desc^+(\hat{F}_q^{(a)}) = F_{pass}. \tag{7}$$

That is to say, a traversal of LNG starting from nodes that correspond to label sets inside $\hat{F}_q^{(a)}$ reaches the exact set of qualified label sets. Further consider a traversal of the unified graph starting from at least one entry vector whose associated label set is L. The combination of cross-group and intra-group proximity edges enables this traversal to exactly reach all vectors whose associated label

set is inside $Desc^+(\{L\})$. As a result, Equation (7) indicates that, the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones, by employing at least one entry vector whose associated label set is L, for every $L \in \hat{F}_q^{(a)}$. Hence, this theorem holds. □

Theorem 2. ApproximateELS *returns a set* $\hat{F}_q^{(a)}$ *of qualified label sets such that*

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}, \tag{8}$$

where $F_{pass} = \{L \in F \mid L_q \subseteq L\}$.

Proof. We prove the theorem by showing the two containment relations. First, we show that $\hat{F}_q \subseteq \hat{F}_q^{(a)}$. If $L_q \in F$, then L_q itself is the only minimum superset of L_q in F. In this case, both the exact ELS retrieval method and ApproximateELS return $\{L_q\}$, and thus the claim holds. Otherwise, when $L_q \notin F$, the exact ELS retrieval method first enumerates candidate label sets whose corresponding trie paths contain all labels in L_q, and then removes non-minimal supersets to obtain the exact ELS set $\hat{F}_q$. In contrast, ApproximateELS keeps these candidate label sets without applying the minimality filtering step. Therefore, every exact ELS is retained by ApproximateELS, which gives

$$\hat{F}_q \subseteq \hat{F}_q^{(a)}. \tag{9}$$

Second, we show that $\hat{F}_q^{(a)} \subseteq F_{pass}$. For any $L \in \hat{F}_q^{(a)}$, ApproximateELS inserts L only after the corresponding trie path has been verified to contain all labels in L_q. Since each terminal node in the trie corresponds to a unique label set in F, this implies

$$L_q \subseteq L. \tag{10}$$

By the definition of F_{pass}, we have $L \in F_{pass}$. Hence,

$$\hat{F}_q^{(a)} \subseteq F_{pass}. \tag{11}$$

Combining Equations 9 and 11, we obtain

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}. \tag{12}$$

Thus, the theorem holds. □

3 XGBOOST ROUTER TRAINING

Sections 4.1-4.2 in the main contents show that different LANNS queries favor different search paradigms. Based on this observation, SODA uses an XGBoost-based router to select among three search paradigms: unified-graph-based search, proximity-graph-based search, and pre-filtering brute-force search. As described in Section 4.3, the router uses four query-level features, i.e., p_{pass}, $|F_{pass}|$, $|L_q|$, and $|\mathbb{I}(l_{\max})|$, which capture both vector selectivity and label-set structure. Moreover, Table 1 shows that all four features receive non-negligible importance scores on real datasets, indicating that each feature contributes to routing decisions. We implement the router with XGBoost because it can capture nonlinear feature interactions while maintaining low inference latency. In our implementation, the maximum tree depth is set to 6, the learning rate is set to 0.05, and the number of boosting rounds is set to 300.

We randomly generate 40,000 LANNS queries from all eight datasets, with 5,000 queries from each dataset. These queries exclude the test queries used in the main algorithmic evaluation. To generate the label of each query, we evaluate the three candidate paradigms using the same implementations and parameter settings as in the main experiments. Specifically, the unified-graph-based paradigm uses UNG+ with $\delta = 6$, the same construction parameter as UNG; the proximity-graph-based paradigm adopts the FAVOR-style search strategy used by SODA; and the pre-filtering paradigm exhaustively computes distances over all qualified vectors, thus always achieving a recall of 1. For each paradigm, we follow the same search-queue-length tuning protocol as in the QPS–recall evaluation, varying the maximum search queue length from 10 to 70,000. We then assign the query to the paradigm with the highest QPS among those whose Recall@10 is at least 0.9. If multiple paradigms have the same QPS after measurement rounding, we break ties by first choosing the one with higher recall, and then by a fixed deterministic order: unified-graph-based search, proximity-graph-based search, and pre-filtering.

Table 1: Feature importance scores.

Data Name	Genome	Reviews	Amazon	VariousImg	Music	BookReviews	Tiktok	Laion
p_{pass}	0.4743	0.4804	0.4835	0.4813	0.4812	0.4809	0.5906	0.4631
$\lvert F_{pass}\rvert$	0.1847	0.1787	0.1137	0.1793	0.1775	0.1130	0.0675	0.1104
$\lvert L_q\rvert$	0.0320	0.0306	0.0687	0.0335	0.0330	0.0698	0.0204	0.0696
$\lvert\mathbb{I}(l_{\max})\rvert$	0.3090	0.3103	0.3341	0.3059	0.3083	0.3363	0.3215	0.3569

For router evaluation, we split 80% queries of each dataset for training and 20% for testing. The model is trained on the combined training queries from all eight datasets and evaluated separately on the test queries of each dataset. We also conduct a leave-one-dataset-out evaluation, where all queries from one dataset are removed from training and the trained model is tested on this held-out dataset. As shown in Tables 6 and 7 in the main contents, the routing accuracy remains above 90% under both settings, indicating that the learned routing rule generalizes across datasets. This accuracy is acceptable, given that the resulting SODA outperforms state-of-the-art methods in main experiments.

As shown in Table 6, this training time cost is only a few seconds. In all algorithm-performance experiments, online latency is measured from receiving the query to returning top-K results, including feature extraction and model prediction. The feature-extraction and prediction overhead is only a few microseconds, showing that the router is suitable for real-time query processing.

4 INDEX CONSTRUCTION COST OF SODA

The proposed SODA mainly constructs two parts of indexes: (i) a unified-graph index for UNG+, including a trie tree and an LNG; and (ii) a proximity-graph of all base vectors. SODA builds indexes within a time complexity of

$$O\Big(P + d_{LNG} \cdot \Big(|F|^2 \cdot |\mathcal{L}| + |V| \cdot p \cdot (d + \log w)\Big)\Big),$$

where $O(P)$ is the cost to build proximity graphs, including the proximity-graph of all base vectors, and proximity graphs on the unified graph; d_{LNG} is the average out-degree of label sets on LNG. The reason is that it takes $O(d_{LNG} \cdot (|F|^2 \cdot |\mathcal{L}| + |V| \cdot p \cdot (d + \log w)))$ to build the unified graph , except inside proximity graphs.

Moreover, the index of SODA has a space complexity of

$$O\Big(X + |F| \cdot (|\mathcal{L}| + d_{LNG} \cdot \delta) + w\Big),$$

where X is the total space cost of proximity graphs; δ is a parameter that determines the number of cross-group edges between two proximity graphs bridged on the unified graph. The reason is that the unified graph has a space cost of $O(|F| \cdot (|\mathcal{L}| + d_{LNG} \cdot \delta) + w)$

, except inside proximity graphs. Furthermore, the router's space overhead is negligible in practice.

5 CACHE HIT RATES COMPARISON

Frequent switching among search paradigms in SODA may reduce cache locality and make repeated data accesses a major performance bottleneck. Therefore, SODA+ groups queries with similar execution behavior to improve memory-access locality. Table 2 reports the cache hit rates of SODA and SODA+, where L1, L2, and L3 denote the three CPU cache levels. SODA+ achieves higher hit rates at all three levels, indicating that query grouping effectively improves cache locality and contributes to its higher throughput.

Table 2: Cache hit rates comparison (SD signifies SODA**).**

Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)	Dataset	Alg.	L1 (%)	L2 (%)	L3 (%)
Genome	SD	89.95	61.49	81.31	Music	SD	88.51	48.82	35.55
	SD+	**89.99**	**83.80**	**91.66**		SD+	**90.53**	**58.61**	**73.98**
Reviews	SD	84.16	44.68	39.38	BookReviews	SD	82.17	60.07	31.41
	SD+	**87.76**	**54.35**	**77.44**		SD+	**88.67**	**60.71**	**74.63**
Amazon	SD	88.79	56.14	27.64	Tiktok	SD	72.88	96.95	18.16
	SD+	**89.84**	**61.57**	**64.11**		SD+	**73.44**	**98.04**	**57.04**
VariousImg	SD	84.32	55.75	18.42	Laion	SD	80.88	59.43	49.95
	SD+	**88.15**	**56.39**	**62.47**		SD+	**86.46**	**65.00**	**71.59**